Notebook 04: Data Cleaning & Panel Construction

Combines raw datasets into state-year panel data (2010–2024), standardizing policy metrics, norm moving averages, and controls.

Important Project Safety Notice

Before executing or citing the findings in this notebook, please read the public guidance on what this project is and is not claiming:

docs/not_saying.md - *What This Theory Is NOT Claiming*

1. Library Imports

We load dependencies, including state_crosswalk and python-dotenv.

In [1]:
```
import os
import sys
import pandas as pd
import numpy as np
from dotenv import load_dotenv

# Add src directory to path
sys.path.append(os.path.abspath('..'))
from src.state_crosswalk import clean_state_code

# Load environment variables
load_dotenv()

os.makedirs('../data/processed', exist_ok=True)
print('Imports and paths verified successfully.')
```

```
Imports and paths verified successfully.
```

2. Base Panel Construction

Initialize a balanced panel of 51 states/regions × 15 years (2010–2024) = 765 observations. Include indicator columns to manage the COVID-era structural break.

In [2]:
```
states = [
    'AL', 'AK', 'AZ', 'AR', 'CA', 'CO', 'CT', 'DE', 'FL', 'GA',
    'HI', 'ID', 'IL', 'IN', 'IA', 'KS', 'KY', 'LA', 'ME', 'MD',
    'MA', 'MI', 'MN', 'MS', 'MO', 'MT', 'NE', 'NV', 'NH', 'NJ',
    'NM', 'NY', 'NC', 'ND', 'OH', 'OK', 'OR', 'PA', 'RI', 'SC',
    'SD', 'TN', 'TX', 'UT', 'VT', 'VA', 'WA', 'WV', 'WI', 'WY', 'DC'
```

```
]
years = list(range(2010, 2025))

panel_records = []
for state in states:
    for year in years:
        panel_records.append({
            'state': state,
            'year': year,
            'covid_era': 1 if year >= 2020 else 0,
            'primary_analysis_window': 1 if year <= 2019 else 0
        })
df_panel = pd.DataFrame(panel_records)
print(f'Base panel created with {len(df_panel)} state-year observations.')
```

```
Base panel created with 765 state-year observations.
```

3. Load and Clean Google Trends Data

Load the Google Trends raw CSV, apply the state crosswalk, and pivot the keyword-specific SVI series into database columns.

In [3]:
```
df_trends_raw = pd.read_csv('../data/raw/google_trends_raw.csv')
df_trends_raw['state_clean'] = df_trends_raw['state'].apply(clean_state_code)
df_trends_raw = df_trends_raw.dropna(subset=['state_clean'])

# Pivot keywords to separate columns
df_trends_pivot = df_trends_raw.pivot_table(
    index=['state_clean', 'year'],
    columns='keyword',
    values='scaled_svi',
    aggfunc='mean'
).reset_index()

# Rename columns to be database-friendly
df_trends_pivot = df_trends_pivot.rename(columns={
    'state_clean': 'state',
    'Common Core': 'svi_common_core',
    'standardized testing': 'svi_testing',
    'opt out testing': 'svi_opt_out',
    'highway funding': 'svi_highway'
})
print('Google Trends data cleaned and pivoted successfully.')
print(df_trends_pivot.head())
```

```
Google Trends data cleaned and pivoted successfully.
keyword state  year  svi_common_core  svi_highway  svi_opt_out  svi_testing
0          AK  2010         0.000000          NaN          NaN          0.0
1          AK  2011         0.613333          NaN          NaN          0.0
2          AK  2012         5.013333          NaN          NaN          0.0
3          AK  2013        16.106667          NaN          NaN          0.0
4          AK  2014        19.253333          NaN          NaN          0.0
```

4. Load and Clean GDELT Data

Load GDELT media salience, clean state codes, filter national/non-state rows, and compute relative salience. We also add a simulated, non-zero highway coverage indicator for the falsification case.

In [4]:
```
df_gdelt_raw = pd.read_csv('../data/raw/gdelt_media_salience.csv')
df_gdelt_raw['state_clean'] = df_gdelt_raw['state_code'].apply(clean_state_code)
df_gdelt_raw = df_gdelt_raw.dropna(subset=['state_clean'])

# Aggregate by state and year
df_gdelt_clean = df_gdelt_raw.groupby(['state_clean', 'year']).agg({
    'total_events': 'sum',
    'edu_events': 'sum',
    'highway_events': 'sum'
}).reset_index().rename(columns={'state_clean': 'state'})

# Compute media salience ratio (matching events / total events)
df_gdelt_clean['gdelt_edu_salience'] = df_gdelt_clean['edu_events'] / df_gdelt_clea
df_gdelt_clean['gdelt_edu_salience'] = df_gdelt_clean['gdelt_edu_salience'].fillna(

# Create actual highway media salience ratio for the falsification case (Pipeline c
# (This uses actual highway events, which is zero in this dataset, but is retained
df_gdelt_clean['gdelt_highway_salience'] = df_gdelt_clean['highway_events'] / df_gd
df_gdelt_clean['gdelt_highway_salience'] = df_gdelt_clean['gdelt_highway_salience']

print('GDELT media salience cleaned successfully.')
print(df_gdelt_clean.head())
```

```
GDELT media salience cleaned successfully.
  state  year  total_events  edu_events  highway_events  gdelt_edu_salience  \
0    AK  2010         81968          21               0            0.000256
1    AK  2011        102194          40               0            0.000391
2    AK  2012        128299          46               0            0.000359
3    AK  2013        130496          59               0            0.000452
4    AK  2014        197020         153               0            0.000777

   gdelt_highway_salience
0                     0.0
1                     0.0
2                     0.0
3                     0.0
4                     0.0
```

5. Load Policy Intensity Index & Standardize Within-Era

Load pre-ESSA NCLB intensity (constructed from waiver dates) and post-ESSA weights ratio. Standardize within-era to prevent structural breaks.

In [5]:
```
# Load actual ESEA flexibility waivers & compile State Policy Intensity Index (0-4)
df_waiver = pd.read_csv('../data/raw/nclb_waivers.csv')
df_essa = pd.read_csv('../data/raw/essa_plan_coding_sheet.csv')
```

```
# Build ESEA waiver active indicator by state-year
df_policy_raw = []
for state in states:
    w_yr = df_waiver[df_waiver['state'] == state]['waiver_approval_year'].values[0]

    # Post-ESSA weights
    essa_row = df_essa[df_essa['state'] == state]
    ach_weight = essa_row['academic_achievement_weight'].values[0] if len(essa_row)

    for year in range(2010, 2025):
        # 1. ESEA waiver active (pre-2018 only, revoked in WA and OK in 2014)
        has_waiver = 0
        if year >= w_yr and year <= 2017:
            if not (state == 'WA' and year >= 2014) and not (state == 'OK' and year
                has_waiver = 1

        # 2. Time-varying State Policy Intensity Index (0-4)
        # Indicator A: High school standardized exit exam (staggered active years)
        exit_exam = 0
        if state in ['MA', 'NJ', 'NY', 'TX', 'FL', 'WA', 'MD', 'OH', 'VA', 'NC']:
            exit_exam = 1
        elif state == 'SC' and year <= 2014:
            exit_exam = 1
        elif state in ['CA', 'GA', 'AZ'] and year <= 2015:
            exit_exam = 1

        # Indicator B: Letter-grade school accountability system (A-F grading, stag
        af_grading = 0
        if state == 'FL':
            af_grading = 1
        elif state == 'TX' and year >= 2018:
            af_grading = 1
        elif state == 'NM' and (year >= 2011 and year <= 2018):
            af_grading = 1
        elif state in ['AZ', 'LA', 'IN'] and year >= 2011:
            af_grading = 1
        elif state in ['OK', 'GA'] and year >= 2012:
            af_grading = 1
        elif state == 'MS' and year >= 2013:
            af_grading = 1
        elif state in ['NC', 'AL'] and year >= 2015:
            af_grading = 1
        elif state == 'AL' and year >= 2018:
            af_grading = 1

        # Indicator C: Third-grade reading test retention law (parent-level stakes,
        third_grade_retention = 0
        if state == 'FL':
            third_grade_retention = 1
        elif state in ['IN', 'CO'] and year >= 2012:
            third_grade_retention = 1
        elif state in ['AZ', 'OH'] and year >= 2013:
            third_grade_retention = 1
        elif state in ['OK', 'NC'] and year >= 2014:
            third_grade_retention = 1
        elif state == 'MS' and year >= 2015:
```

```
            third_grade_retention = 1
        elif state == 'MI' and (year >= 2020 and year <= 2022):
            third_grade_retention = 1
        elif state == 'TN' and year >= 2023:
            third_grade_retention = 1

        # Indicator D: Test-linked teacher evaluations (VAM evaluation linkage)
        # Pre-ESSA, this was mandated by waivers. Post-ESSA, it was rolled back in
        vam_eval = 0
        if year <= 2017:
            vam_eval = 1 if has_waiver == 1 else 0
        else:
            # Post-ESSA: VAM remained active in high-achievement-weight states
            vam_eval = 1 if state in ['FL', 'TX', 'NC', 'OH', 'TN', 'AZ', 'LA'] els

        raw_intensity = exit_exam + af_grading + third_grade_retention + vam_eval

        df_policy_raw.append({
            'state': state,
            'year': year,
            'has_waiver': has_waiver,
            'exit_exam': exit_exam,
            'af_grading': af_grading,
            'third_grade_retention': third_grade_retention,
            'vam_eval': vam_eval,
            'raw_intensity': raw_intensity
        })

df_policy_raw = pd.DataFrame(df_policy_raw)

# Standardize overall (or within-era to maintain scale comparability)
df_policy_raw['policy_intensity'] = 0.0
for mask in [df_policy_raw['year'] <= 2017, df_policy_raw['year'] >= 2018]:
    mean_val = df_policy_raw.loc[mask, 'raw_intensity'].mean()
    std_val = df_policy_raw.loc[mask, 'raw_intensity'].std()
    if std_val == 0 or pd.isna(std_val): std_val = 1.0
    df_policy_raw.loc[mask, 'policy_intensity'] = (df_policy_raw.loc[mask, 'raw_int

print('State Policy Intensity Index compiled and standardized with staggered adopti
```

```
State Policy Intensity Index compiled and standardized with staggered adoption date
s.
```

6. Calculate EWMA-Based Adaptive Norm (N_s,t)

Calculate $N_{s,t}$ recursively using an Exponentially Weighted Moving Average (EWMA) with $ u=0.08$ to match the simulation's dynamic feedback loop.

In [6]:
```
def calculate_ewma_norm(df, nu=0.08):
    df = df.sort_values(by=['state', 'year']).copy()
    norms = []
    for state, group in df.groupby('state'):
        group = group.sort_values('year')
        p_vals = group['policy_intensity'].values
```

```
        n_vals = np.zeros(len(p_vals))
        n_vals[0] = p_vals[0]
        for t in range(1, len(p_vals)):
            n_vals[t] = nu * p_vals[t-1] + (1 - nu) * n_vals[t-1]
        group['norm'] = n_vals
        norms.append(group)
    return pd.concat(norms, ignore_index=True)

df_policy = calculate_ewma_norm(df_policy_raw, nu=0.08)
print('EWMA-based norm calculated successfully.')
```

```
EWMA-based norm calculated successfully.
```

7. Merge Datasets & Construct Disaggregated Backlash Indexes

Merge all datasets into our skeleton panel and construct disaggregated backlash indices (B_{media} and B_{mass}) to keep elite signals and mass mobilization separate.

In [7]:
```
df_merged = df_panel.merge(df_policy, on=['state', 'year'], how='left')
df_merged = df_merged.merge(df_trends_pivot, on=['state', 'year'], how='left')
df_merged = df_merged.merge(df_gdelt_clean, on=['state', 'year'], how='left')

# Fill NaNs
for col in ['svi_common_core', 'svi_testing', 'svi_opt_out', 'svi_highway', 'gdelt_
    df_merged[col] = df_merged[col].fillna(0.0)

# Standardize backlash inputs
for col in ['gdelt_edu_salience', 'svi_common_core', 'svi_testing', 'svi_opt_out']:
    df_merged[f'{col}_std'] = (df_merged[col] - df_merged[col].mean()) / df_merged[

# 1. Elite/Media Backlash Index (Standard, Winsorized, and Logged)
df_merged['backlash_media'] = df_merged['gdelt_edu_salience_std']
df_merged['backlash_media_winsorized'] = df_merged['backlash_media'].clip(lower=-3.
df_merged['gdelt_edu_salience_logged'] = np.log1p(df_merged['gdelt_edu_salience'] *

# 2. Mass Mobilization Backlash Index
df_merged['backlash_mass'] = (df_merged['svi_common_core_std'] + df_merged['svi_tes

print('Merged state-year panel constructed with outlier handling.')
```

```
Merged state-year panel constructed with outlier handling.
```

8. Partisan, Electoral & Spell Controls

Construct gubernatorial election cycle covariates, governor party changes, and uninterrupted single-party control spells to test the pendulum against simple electoral turnover.

In [8]:
```
# Load actual state political control panel data
df_political = pd.read_csv('../data/raw/state_political_controls.csv')
```

```
# Merge political controls into our clean panel
df_merged = df_merged.merge(df_political, on=['state', 'year'], how='left')

# Calculate governor party changes and single-party spells
df_merged = df_merged.sort_values(by=['state', 'year']).copy()
party_changes = []
spells = []

for state, group in df_merged.groupby('state'):
    group = group.sort_values('year')
    parties = group['gov_party_rep'].values
    changes = [0]
    current_spell = [1]

    for i in range(1, len(parties)):
        change = 1 if parties[i] != parties[i-1] else 0
        changes.append(change)

        if change == 1:
            current_spell.append(1)
        else:
            current_spell.append(current_spell[-1] + 1)

    group['gov_party_change'] = changes
    group['within_party_stability'] = current_spell
    df_merged.loc[group.index, 'gov_party_change'] = changes
    df_merged.loc[group.index, 'within_party_stability'] = current_spell

# Add simulated/approximated legislative trifectas (if missing, but state_political
print('Actual political control variables merged.')
```

```
Actual political control variables merged.
```

9. Highway Funding Falsification Case

Build simulated highway policy intensity based on FHWA apportionments per lane-mile and transportation search volumes. Standardize within-era and compute its EWMA norm.

In [9]:
```
# Load actual federal highway funding apportionments
df_highway_policy = pd.read_csv('../data/raw/highway_apportionments.csv')

# Standardize highway funding within-era for the control case
df_highway_policy['highway_policy_intensity'] = 0.0
for mask in [df_highway_policy['year'] <= 2017, df_highway_policy['year'] >= 2018]:
    mean_val = df_highway_policy.loc[mask, 'raw_highway_intensity'].mean()
    std_val = df_highway_policy.loc[mask, 'raw_highway_intensity'].std()
    if std_val == 0 or pd.isna(std_val): std_val = 1.0
    df_highway_policy.loc[mask, 'highway_policy_intensity'] = (df_highway_policy.lo

# Calculate highway norm using identical EWMA logic
df_highway_policy = calculate_ewma_norm(df_highway_policy.rename(columns={'highway_
df_highway_policy.rename(columns={'policy_intensity': 'highway_policy_intensity', '

# Merge highway funding controls into our panel
```

```
df_merged = df_merged.merge(df_highway_policy[['state', 'year', 'raw_highway_intens

# Standardize real highway backlash inputs
mean_val_g = df_merged['gdelt_highway_salience'].mean()
std_val_g = df_merged['gdelt_highway_salience'].std()
if std_val_g == 0 or pd.isna(std_val_g): std_val_g = 1.0
df_merged['gdelt_highway_salience_std'] = (df_merged['gdelt_highway_salience'] - me

mean_val_s = df_merged['svi_highway'].mean()
std_val_s = df_merged['svi_highway'].std()
if std_val_s == 0 or pd.isna(std_val_s): std_val_s = 1.0
df_merged['svi_highway_std'] = (df_merged['svi_highway'] - mean_val_s) / std_val_s
df_merged['highway_backlash_media'] = df_merged['gdelt_highway_salience_std']
df_merged['highway_backlash_media_winsorized'] = df_merged['highway_backlash_media'
df_merged['gdelt_highway_salience_logged'] = np.log1p(np.maximum(0, df_merged['gdel
df_merged['highway_backlash_mass'] = df_merged['svi_highway_std']

# Define main backlash and highway backlash variables
df_merged['backlash'] = df_merged['backlash_media_winsorized']
df_merged['highway_backlash'] = df_merged['highway_backlash_media_winsorized']

print('Actual highway control case variables merged and backlash columns defined.')
```

```
Actual highway control case variables merged and backlash columns defined.
```

10. Export Panel & Run Diagnostic Check

Save the final processed panel to `data/processed/state_year_panel.csv` and execute boundary correlation diagnostics.

In [10]:
```
df_merged.to_csv('../data/processed/state_year_panel.csv', index=False)
print('Clean panel exported to data/processed/state_year_panel.csv.')

# Boundary correlation diagnostic check (P_2017 vs P_2018)
p_2017 = df_merged[df_merged['year'] == 2017].set_index('state')['policy_intensity'
p_2018 = df_merged[df_merged['year'] == 2018].set_index('state')['policy_intensity'
correlation = p_2017.corr(p_2018)
print(f'Rank-order correlation of policy intensity across 2017/2018 boundary: {corr
```

```
Clean panel exported to data/processed/state_year_panel.csv.
Rank-order correlation of policy intensity across 2017/2018 boundary: 0.890
```